

Interview Assignment: Build an AI-Assisted Software Engineering System — Audit Log Service

Charles Schwab & Co., Inc. — Confidential & Proprietary. Provided solely for individual candidate assessment. Do not copy, distribute, re-host, or retain after submission. **Version: 2.0** | **Date: 2026-08-03**

0. How to Submit & Integrity Expectations — Read First

This is an individual, confidential assessment. We evaluate the system you build **and** the evidence that you built it yourself. Please read and follow these expectations; they are part of how the submission is assessed.

0.1 Submit via a Private GitHub Repository

- Do your work in a **Git repository from the start** and push it to a **private GitHub repository**. Grant the panel read access when you submit (we will provide the reviewer handles).
- Develop **in the open in your own repo**: commit your work as you go so the repository reflects how you actually built the solution, from requirement analysis through design, implementation, and validation.
- Commit under **your own GitHub identity**, and keep your AI usage log connected to the work in your repository.
- **Submit the repository, not a snapshot**. A zip/tarball of final files, or a repository with no development history, does not meet the bar for this exercise.

0.2 Do Your Own Work, on Your Own Setup

- Complete the assignment **individually, on your own machine and under your own accounts**.
- The submission must be **your own original work**. Do not start from, copy, or share another person's solution, and do not build on a shared or jointly-accessed copy of this assignment.
- Please keep this assignment, the problem, and your solution **confidential** — do not forward, re-host, or distribute them. This material is Charles Schwab confidential and proprietary.

0.3 AI Use Is Expected — Just Be Honest About It

Using AI (Copilot / Claude / etc.) is the whole point of this exercise, and we want to see it. The only thing we ask is that your submission **honestly represents your own process and authorship** — that you can explain and defend what you built and how you used AI to build it.

0.4 Attestation (required)

Add an `ATTESTATION.md` file to the root of your repository. Record your **full name**, your **email address**, the **assignment title** you were given, and the **dates** you started and submitted, followed by this statement:

I, [your full name], attest that this submission is my own individual work, completed on my own machine and accounts, and that it honestly reflects my development process and use of AI.

1. Objective

Build a working prototype that transforms a set of requirements into a reviewable engineering outcome using AI-assisted engineering execution. Demonstrate requirement understanding, task decomposition, multi-step autonomous orchestration. Focus on engineer-led execution accelerated by AI, not autonomous orchestration.

2. Scenario

You will build a **tamper-evident audit log service** — a system that records an append-only history of events and guarantees that past records cannot be modified or deleted without detection.

Your task is to design and build it over **2–3 days** using AI assistance (Copilot / Claude / etc.) while demonstrating engineering judgment at every step.

3. Scope

- Greenfield scenarios (new systems / features)
- Feature extension on your own codebase
- Test and documentation improvements
- Well-defined and ambiguous requirements

4. Core Requirements

1. **Requirement Understanding:** Interpret intent, identify ambiguity, normalize into a clear engineering problem.
2. **Task Decomposition:** Convert high-level requirements into actionable tasks with dependencies and sequencing.
3. **AI-Assisted Execution (Critical Differentiator):** Use AI across implementation, debugging, refactoring, test generation, documentation, and review preparation.
 - Define tasks with intent, constraints, acceptance criteria, and technical context.
 - Use disciplined prompting with iterative refinement.
 - Maintain traceability of AI use (generated / edited / rejected, with rationale) within your repository.
 - Apply quality gates (analysis, linting, tests, security, performance).
 - Enforce secure AI usage.
 - Require human sign-off for high-impact changes.
 - Retain explicit engineer ownership of correctness, maintainability, and production readiness.
4. **Engineering Output Generation:** Produce production-quality code, API / schema definitions, unit / integration tests, and supporting documentation with clean design and maintainability.
5. **Validation and Risk Control:** Identify risks, trade-offs, and failure scenarios; define validation and safety guardrails.
6. **Controlled Oversight:** Engineer leads execution and approves all outputs; AI assists within tasks.
7. **Final Engineering Summary:** Include plan / rationale, artifacts, risks / trade-offs / validation, assumptions, and limitations.

5. Scenario Details

Scenario A — Greenfield: Core Audit Log Service

Build an audit log service with the following capabilities:

Write API Accept an event record containing at minimum:

- `eventType` — what happened (e.g., `USER_LOGIN`, `RECORD_UPDATED`, `PERMISSION_GRANTED`)
- `actorId` — who or what caused the event
- `resourceType` — the type of resource affected
- `resourceId` — the specific resource affected
- `payload` — a structured object with event-specific detail
- `timestamp` — when the event occurred (caller-supplied or server-assigned; document your choice)

Records are **append-only**: the API must not expose an update or delete operation.

Query API Retrieve events with filtering by any combination of:

- `actorId`
- `resourceType` and `resourceId`

- `eventType`
- Time range (`from` / `to`)

Support pagination for large result sets.

Tamper Evidence — Hash Chain Each stored record must include:

- A hash of its own content (the event fields above)
- A hash of the immediately preceding record (or a defined genesis value for the first record)

Together these form a **hash chain**: any modification to a past record invalidates its own hash and every hash that follows it, making tampering detectable.

Chain Verification Endpoint Expose a `GET /audit/verify` endpoint that walks the full chain and reports:

- Whether the chain is intact
- If broken: which record is the first inconsistency and what type of violation was detected

The entire assignment is validated through these APIs — write events, query them, verify the chain, then modify a record directly in the data store and verify again to confirm detection. No external application or consumer is required.

Scenario B — Extend Your Own System: Retention and Redaction

Extend the service you built in Scenario A with:

Retention Policy Records older than a configurable window should be archivable or soft-deletable. The chain verification endpoint must handle the presence of archived records correctly and not report a false positive break for records that were legitimately archived per policy.

Structured Redaction Certain fields within a record's payload may contain sensitive data (e.g., account numbers, personal identifiers) that must be redactable to satisfy data privacy requirements — without breaking the hash chain.

This is a genuine engineering problem: the original hash covers the original value, so simply removing the value would invalidate the hash. Design and implement a redaction scheme that satisfies both tamper-evidence and data privacy. Document your approach, the trade-offs you considered, and any limitations of your chosen solution.

Bulk Export Provide an endpoint to export all records for a given `resourceId` or `actorId` as a self-contained, verifiable bundle. The bundle must include enough chain metadata for a recipient to independently verify the records it contains have not been altered since export.

Scenario C — Ambiguous: Compliance Reporting

Product says: *"Regulators need to be able to audit access to client account data."*

This requirement is **intentionally under-specified**. Demonstrate:

- How you clarify and normalize the requirement before writing any code.
- What ambiguities you identified and what assumptions you made (or what questions you would ask before proceeding).
- How you translate the clarified requirement into a concrete technical design.
- What you chose to implement versus what you scoped out, and why.

Your submission must include the clarified requirement statement you worked from, the design decisions it produced, and the implementation (or a well-reasoned partial implementation with a documented scope boundary).

6. Live Defense (Scheduled After Submission)

After you submit, you will join a **live review session** with the panel to walk through your solution, explain and defend design and AI-usage decisions the panel asks about, and work through a small requirement change live in your own codebase. Come with your environment ready to run and modify the code.

7. Deliverables

All delivered in your **private GitHub repository**:

- **The repository itself** — shared with the panel, containing your development history (this is how you submit; not a zip or a snapshot).
- **ATTESTATION.md** — the attestation from §0.4.
- **Working prototype** — runnable end-to-end, with setup instructions.
- **Architecture overview** — components, data model, API design, key decisions, and trade-offs (including hash algorithm choice and chain design).
- **Three scenarios** — each showing decomposition, execution, and validation (A, B, and C).
- **Setup instructions** — how to run locally; any dependencies or prerequisites.
- **Testing approach, limitations, and trade-offs** — what is covered, what is not, and why.
- **AI usage log / traceability notes** — what was prompted, what was accepted / modified / rejected, and why.
- **Final engineering summary** — plan / rationale, artifacts, risks / trade-offs, assumptions, and limitations.

8. Evaluation Criteria (High Level)

Your work is scored against a detailed reviewer rubric. At a high level we assess: engineering reasoning and ambiguity management; system design and correctness; effective, well-governed AI-assisted execution; the authenticity and ownership of your work; code quality; testing and validation rigor; security and production readiness; how you defend and adapt your solution live; and communication. Specific weightings and criteria are not published in candidate materials.

We reward genuine engineering judgment you can explain and defend — not artifacts produced to match a checklist.

9. Expectation

Treat this as production-grade engineering work. Demonstrate strong design fundamentals, effective AI use as an accelerator, output ownership, and defensible reasoning — with an authentic, verifiable development process behind it.

Principle: AI assists the engineer within tasks; the engineer owns execution, quality, and authorship.

Charles Schwab & Co., Inc. — Confidential & Proprietary. Provided solely for individual candidate assessment; do not copy, distribute, re-host, or retain after submission.